SLAM performance predicition

Mauro Tellaroli
Statistical Methods for Maching Learning
Università degli Studi di Milano

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1 Introduction

In the field of autonomous mobile robot, the Simultaneous Localization And Mapping (SLAM) represents one of the most significant problems. As the name suggests, the goal of the SLAM is incrementally obtain from a mobile robot a map of the environment and simultaneously locate the robot's position inside it. The difficulty of the SLAM problem is that to precisely localize itself, the robot needs an accurate map, while to obtain an accurate map, it needs an accurate localization inside the map. This type of problems is called chicken-and-egg problem.

1.1 SLAM methods

In the last 30 years several approaches have been developed to solve the SLAM problem leading to methods extensively used in both common and industrial scenarios. The main paradigms [1] used are:

- Extended Kalman Filter: a vector containing the estimates of the robot position and the environment landmark is used. In addition, a matrix composed of the correlations between the positions and landmark estimates is also used. Both the vector and the matrix are updated using the extended Kalman Filter.
- Particle Filter: the Monte Carlo method is used to sample the possible states of the robot. A random sample is called a particle and it's composed of the robot position and a map estimate. The main steps are: predict the robot's pose for each particle, update particle weights based on sensor data, resample particles according to their weights, and update each particle's map.
- Graph-based: a graph is used where nodes represent robot poses or landmarks, and edges represent spatial constraints derived from sensor observations or odometry. The SLAM problem is formulated as a graph optimization task, where the goal is to find the configuration of nodes that best satisfies all constraints, typically using nonlinear optimization techniques.

1.2 Accuracy of SLAM methods

The heterogeneity of SLAM methods makes it difficult to find a measure that can evaluate their godness. Many approaches rely on manual evaluation or utilize information derived from the specific algorithm. Nowadays the most used metric is the Absolute Pose Error (APE)[2]. The APE measures the difference between the estimated trajectory produced by the SLAM algorithm and the ground truth trajectory, focusing on the global consistency of the estimated poses. The big advantage of the APE is that it allows for a quantitative assessment of how well the SLAM algorithm reconstructs the robot's trajectory over time, independent of the internal workings of the specific method used.

The APE is composed of two sub-metrics:

1. The Absolute Translational Error (ATE): it measures the difference in position, typically using the Euclidean distance between corresponding poses.
2. The Absolute Rotational Error (ARE): it measures the difference in orientation, often computed as the angular distance between corresponding rotations.

Figure 1 and 2 show respectively the ATE variation over time and the difference between the ground truth and the robot estimated trajectory during an environment exploration.

1.3 APE calculation

As anticipated in the previous section, the APE metric is based on the difference between the estimated robot poses and the corresponding ground truth poses.

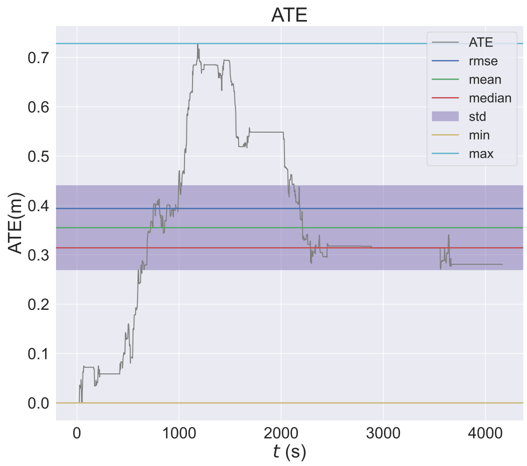

Figure 1: ATE variation over time

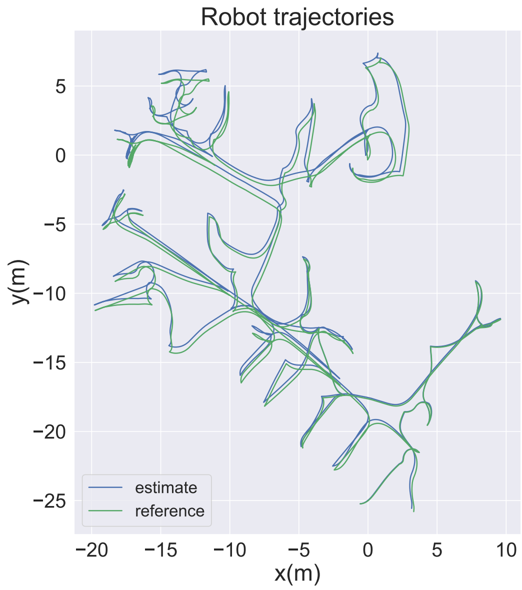

Figure 2: Difference between real (in green) and estimated (in blue) trajectory

Formally, let $\boldsymbol{x}_{1:T}$ be the poses of the robot estimated by a SLAM algorithm from time step 1 to T, and let $\boldsymbol{x}^*_{1:T}$ be the corresponding ground truth poses.

Let $\delta_{i,j}$ be the relative transformation that moves the robot from pose x_i to x_j:

$$\delta_{i,j} = x_j \ominus x_i$$

$$\delta^*_{i,j} = x^*_j \ominus x^*_i$$

Let $trans(\cdot) \in \mathbb{R}^2$ be the function that extracts the translational (position) component from a transformation, and let $rot(\cdot) \in \mathbb{R}^2$ extract the rotational (orientation) component as Euler angle.

Finally, to compute the APE:

$$\text{APE} = \underbrace{\frac{1}{N}\sum_{i,j} trans(\delta_{i,j} \ominus \delta^*_{i,j})^2}_{\text{ATE}} + \underbrace{\frac{1}{N}\sum_{i,j} rot(\delta_{i,j} \ominus \delta^*_{i,j})^2}_{\text{ARE}}$$

1.4 Project goal

Typically, the APE is calculated a posteriori, after the robot has explored the entire environment. The main goal of this project is to propose a predictor for the SLAM performance, specifically the APE, before running the algorithm or during its execution, using features derived from the input data or intermediate results. This type of prediction can improve the robustness and efficiency of autonomous mobile systems since the position error (APE) can be used to correct the robot trajectory.

The work in [3] proposes two main APE predictors, one based on linear regression, and another one based on a Gaussian Process via an RBF Kernel. Although Gaussian Process regression showed slightly better performance compared to linear regression model, the authors preferred linear regression because they considered it a more interpretable model. In both cases, a feature extractor is used [4] obtaining a set of structural features to represent the environment as a vector, which is then passed to the model.

2 Data

2.1 Dataset composition

The dataset is composed of 4613 datapoints. Each datapoint is represented as a tuple

$$(M, a, ate, are)$$

where:

1. $M \in [0, 255]^{n \times n}$: floorplan images representend as grayscale maps that a robot has obtained during its navigation. Some examples are reported in Figure 3;
2. $a \in \mathbb{R}^+$: area measurements of the floorplan images, expressed in m^2;
3. $ate \in \mathbb{R}^+$: ATE values, expressed in m;
4. $are \in \mathbb{R}^+$: ARE values, expressed in rad;

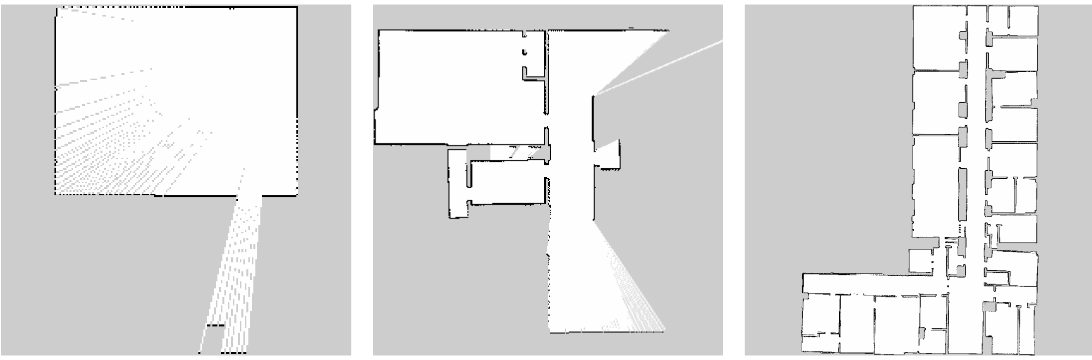

Figure 3: Examples of grayscale floorplan images used as input; the white pixels represent the free area, the black pixels the obstacles and the gray pixels the unknown area.

2.2 Data preprocessing

Evaluating the APE values shows that they are mainly distributed towards zero, with a significant presence of outliers, especially for ATE values. Figure 4 shows the box plot with highlighted the 0.99-quantile and the first value such that its Z-score is greater or equal to 3.5; the Z-score of a number x is $Z = \frac{x - \overline{x}}{\sigma}$ where $\overline{x}$ is the mean and σ the standard deviation.

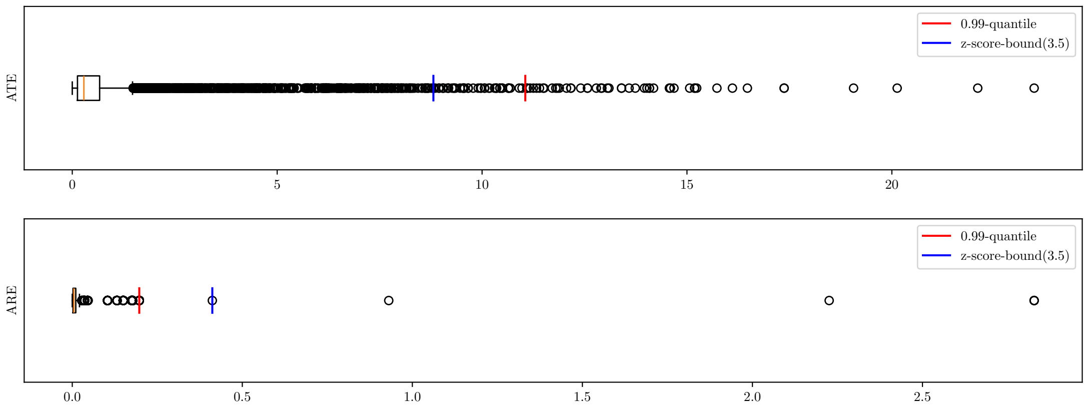

Figure 4: Box plot of the ATE (top) and ARE (bottom) values; is also shown the 0.99-quantile (red line) and the first value with a z-score equal to 3.5 (blue line).

To facilitate our models to learn each quantity, the following strategies have been applied:

- Outliers cleaning: data points with ATE or ARE values above the 0.99-quantile or with a Z-score greater than or equal to 3.5 have been removed to reduce the impact of extreme outliers. 113 datapoints were removed;

- Logarithmic rescaling: due to the skewed distribution of ATE and ARE, a logarithmic transformation has been applied to these values to reduce the effect of large values and improve model learning; let x be the interested value to be rescaled, the applied operation is:

$$\log(1 + x)$$

  where log is the natural logarithm;

- Rescaling: after the logarithmic rescaling, a linear rescaling to $[0, 1]$ has been applied; let $\{x_1, \ldots, x_n\}$ be interested values to be rescaled, the applied operation is

$$\frac{x_i}{\max\limits_{1 \leq k \leq n} x_k} \qquad \text{i=1,...,n}$$

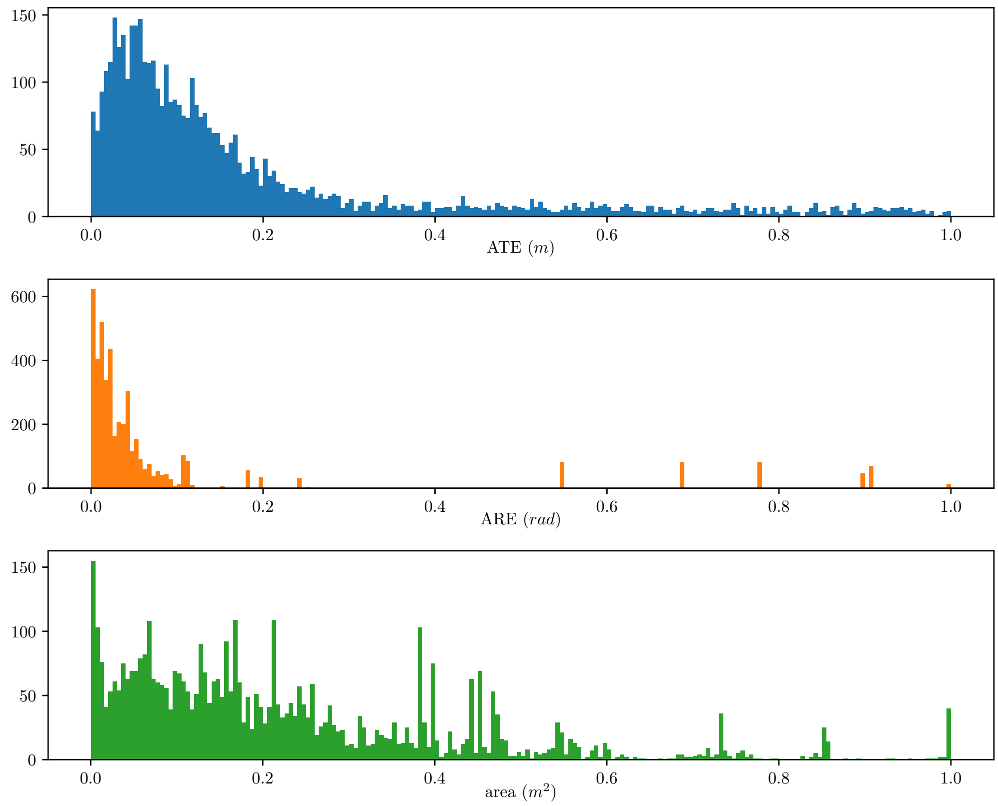

Figure 5: Dataset values after the log and linear rescaling.

- Image cropping: each map has been cropped to remove unnecessary borders, focusing only on the relevant area to reduce input size and eliminate irrelevant background; the crop is always squared in order to mantain this property to all floorplan images. Figure 6 shows an example of this operation;

Figure 6: A floorplan (left) and its bounding square crop (right).

- Image resizing: each floorplan image is resized to 400×400 pixels to ensure a uniform input size for the neural network models;

2.3 Data augmentation

Given the relatively limited size of the dataset, the following data augmentation techniques have been applied to training set:

- Random flip: both horizontal and vertical flips are applied;
- Random rotation: random rotation between $0°, 90°, 180°, 270°$ is applied;

3 Models

In this project, different architectures are used, all based on Convolutional Neural Networks (CNNs) to delegate feature extraction to the model, directly using the environment floorplan image as input. Three distinct models were used:

1. AlexNet [5], one of the first successful architectures in the field of CNNs. It will be a baseline for the other models;
2. MobileNetV3 [6], the state of the art of lightweight CNN architectures designed for mobile and embedded vision applications;
3. SqueezeNet [7], an ultra-compact CNN with few parameters, offering decent accuracy for low-memory device;

Formally, the model h we aim to learn can be expressed as follows:

$$h : [0, 255]^{n \times n} \times \mathbb{R}^+ \to \mathbb{R}^2$$

where $[0, 255]^{n \times n}$ is a floorplan image encoded as an $n \times n$ grayscale image, $\mathbb{R}^+$ is the area of the floorplan expressed in m^2, and $\mathbb{R}^2$ contains ATE (m) and ARE (rad).

3.1 AlexNet

To begin, a model based on AlexNet was chosen, one of the first successful architectures in the field of convolutional neural networks. AlexNet consists of several convolutional layers followed by fully connected layers, using ReLU activations and max-pooling. The relatively large number of layers without the use of modern techniques to lighten its computational load, makes this model a deep CNN resulting in a high number of parameters (37 298 882). The abstract structure is shown on Figure 7.

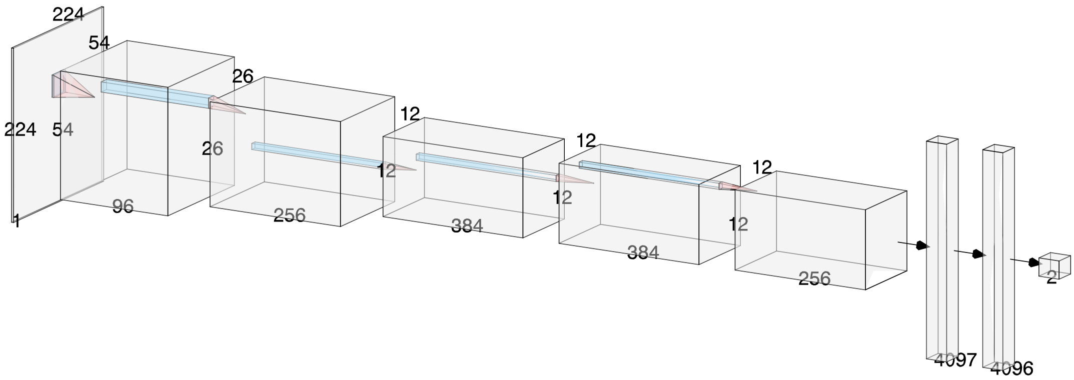

Figure 7: Model structure based on AlexNet. Realized with [8].

Two main sections can be outlined:

1. A CNN feature extractor: it processes the input image through a series of convolutional and pooling layers, resulting in a condensed set of features;
2. A Regression head: it predicts ATE and ARE values through some fully connected layers whose input is the set of features extracted from the CNN;

Note that the first layer of the regression head has size 4097 instead of 4096, since the area of the floorplan is added as an input concatenated to the feature vector extracted by the CNN. This is because otherwise the scaling done by the model in the initial stage would cause the true value of the area to be lost. This procedure was done on all three proposed models.

3.1.1 Dropout

To reduce overfitting, the dropout technique [9] is applied within the fully connected layers of the regression head. Dropout randomly sets a fraction of the input units to zero during training, which prevents the network from relying too heavily on specific features and encourages the learning of more robust representations. In our implementation, a dropout rate of 0.5 is used after each fully connected layer.

3.2 MobileNetV3

Despite AlexNet's good accuracy, the computational requirements it needs make it unsuitable for hardware used in mobile robotics. For this reason, MobileNetV3 was selected as a more efficient alternative, offering a state-of-the-art trade-off between accuracy and computational efficiency. In particular, the MobileNetV3 small version was used, which has 1 520 642 parameters, 24 times less than AlexNet.

To achieve this result, different optimization techniques are used, but we will focus only on the main ones.

3.2.1 Depthwise and Pointwise Convolutions

Convolutional layers on which CNNs are based have led to massive achievements in machine learning projects, especially in computer vision. However, standard convolutions are computationally expensive, particularly for deep networks. MobileNetV1 addresses this by employing depthwise separable convolutions, which split the convolution operation into two parts:

1. Depthwise convolution: processes each input channel separately with its own filter, extracting spatial features independently for each channel;
2. Pointwise convolution: applies 1×1 convolutions across all channels, combining the outputs from the depthwise step to integrate information across channels;

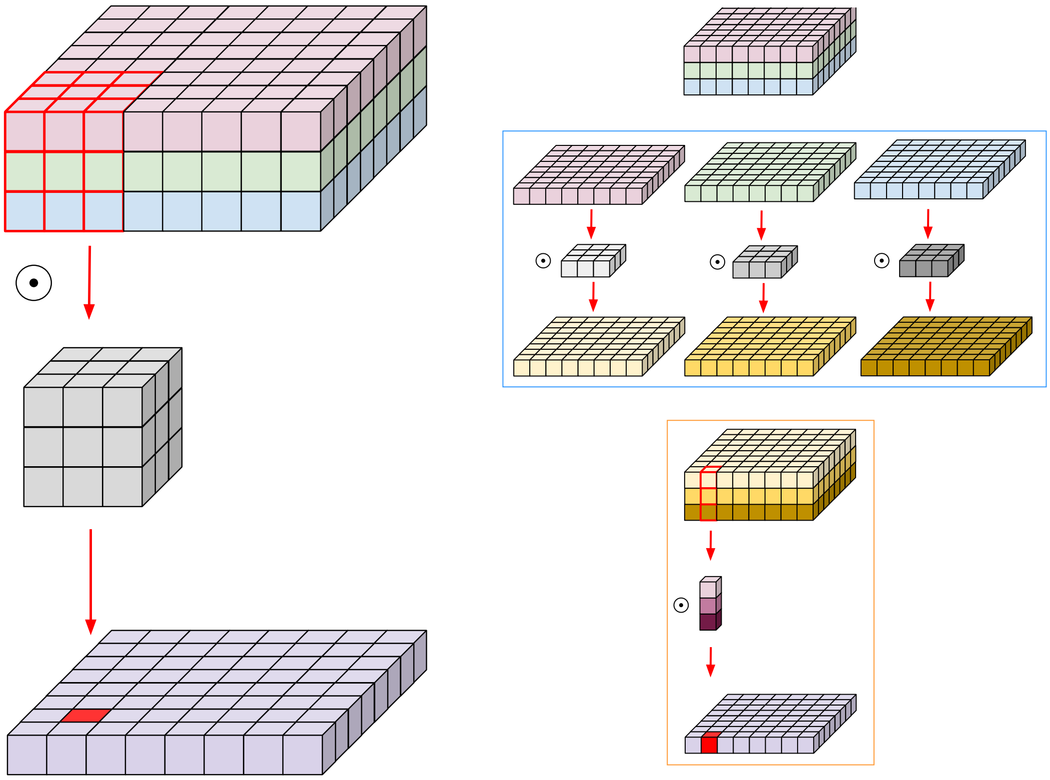

Figure 8: Standard Convolution (left) and Deepthwise+Pointwise Convolution (right). The Deepthwise conv. is highlighted in light blue while Pointwise conv. in orange [10].

To understand computational savings, take the follwing example: let S be the dimensions of the squared input, K the dimensions of the squared kernel filter, C_{in} the number of input channels and C_{out} the numbers of output channels. We'll assume:

$$S = 128 \quad K = 3 \quad C_{in} = 3 \quad C_{out} = 16$$

	Regular convolution	Deepthwise and Pointwise convolution
Parameters	$3^2 \cdot 3 \cdot 16 = 432$	$3^2 \cdot 3 \cdot 128^2 \cdot 16 = 7\,077\,888$
# multiplications	$3^2 \cdot 3 + 3 \cdot 16 = 75$	$3^2 \cdot 3 \cdot 128^2 + 128^2 \cdot 3 \cdot 16 = 1\,228\,800$

3.2.2 Squeeze and Excitation

Introduced in [11], Squeeze and Excitation is a technique that focuses on the interdependencies between channels. As the name suggests, there are two phases:

1. Squeeze phase: each channel is squeezed into a single value through the use of averge pooling. The output of this phase is then a vector $\boldsymbol{z}$ of C elements, where C is the number of channels:

$$\boldsymbol{z} = \{z_1, \dots, z_C\}$$

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j) \qquad \text{c=1,...,C}$$

   where $\boldsymbol{U} = \{\boldsymbol{u}_1, \dots, \boldsymbol{u}_C\}$ is the set of feature maps of size $H \times W$ and $u_c(i,j)$ is the element in position (i,j) on the c-th feature map.

2. Excitation phase: the squeezed vector $\boldsymbol{z}$ is fed into a two-layer feed-forward neural network composed of a first layer of size C/r with ReLU and a second layer of size C with sigmoid,

where $\text{sigmoid}(x) = 1/(1 + e^{-x})$ and r is a reduction ratio. The purpose of this phase is to capture the intricate dependencies among channels. The output of this network is another vector $\boldsymbol{s}$ of the same dimensions of $\boldsymbol{z}$, containing the importance weights for each channel.

3. Scale and combine phase: the importance weights vector $\boldsymbol{s}$ is used to rescale each channel of the original input by channel-wise multiplication ($*$):

$$\tilde{\boldsymbol{x}}_c = s_c * \boldsymbol{u}_c \qquad \text{c=1,}\ldots\text{,C}$$

where $\tilde{\boldsymbol{X}} = \{\tilde{\boldsymbol{x}}_1, \ldots, \tilde{\boldsymbol{x}}_C\}$ is the final output. In this way most important feature are highlighted and the less important ones are downplayed.

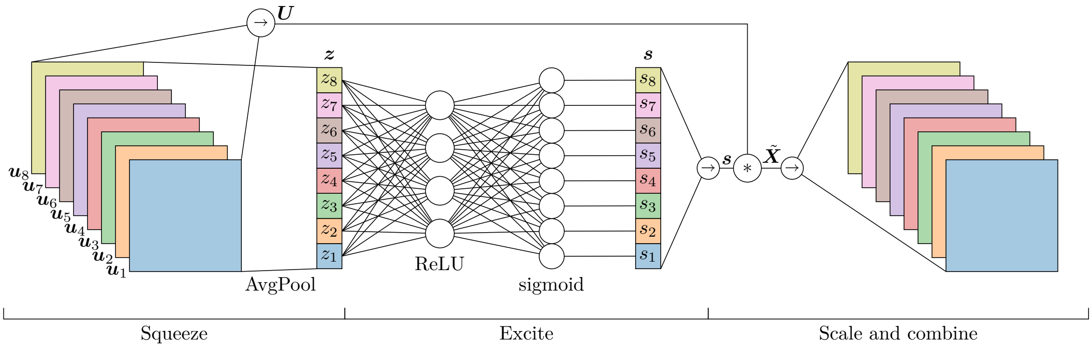

Figure 9: A graphical representation of Squeeze and excitation; in this case a reduction ratio of $r = 2$ is used with 8-channel feature maps.

3.2.3 HardSwish Activation function

To improve the accuracy of the neural networks ReLU was replaced by a new activation function called *swish*, defined as:

$$\text{swish } x = x \cdot \sigma(x)$$

where σ is the sigmoid function $\sigma(x) = 1/(1+e^{-x})$. However, despite swish improves accuracy, it is computationally expensive for mobile devices. To address this, MobileNetV3 introduces the *HardSwish* activation function, which approximates swish while being more efficient to compute. HardSwish is defined as:

$$\text{h-swish}(x) = x \cdot \frac{\text{ReLU6}(x + 3)}{6}$$

where $\text{ReLU6}(x) = \min(\max(0, x), 6)$. This function retains the benefits of swish but is faster and more suitable for deployment on resource-constrained hardware.

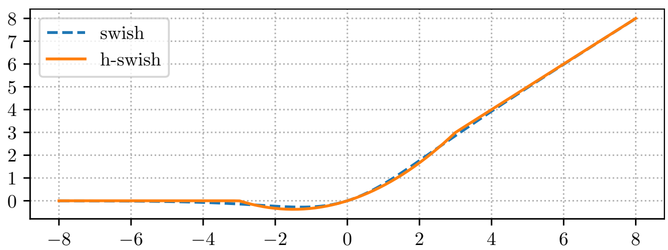

Figure 10: Swish and h-swish

3.3 SqueezeNet

As final model SqueezeNet was choosen. SqueezeNet is an ultra-compact convolutional neural network architecture designed to achieve AlexNet-level accuracy with significantly fewer parameters, as the original paper title “AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size” [7] suggests; as proof of this, the model that was used, derived from a minor modification of SqueezeNet v1.1, has 722 740 params, ∼51.6 times less than AlexNet models described in section 3.1. The few parameters of SqueezeNet also led to a large reduction in power consumption and inference time, making it one of the most widely used architectures in various fields such as autonomous driving, IoT, and mobile AI.

Another aspect of SqueezeNet is the absence of a Fully Connected (FC) layer; this is simply because these kind of layers use a large number of parameters. In our model, shown in Figure 11, a small FC layer was added so that the area value was also utilized. This addition resulted in a very slight increase in the model parameters, precisely 362.

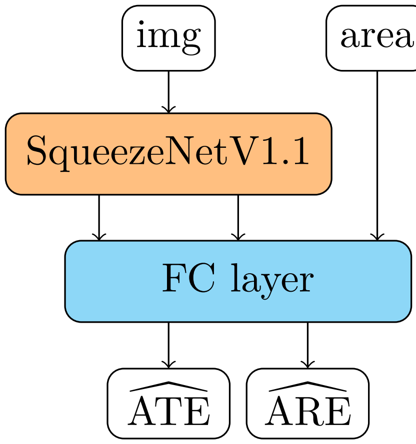

Figure 11: Model structure based on SqueezeNet v1.1

To understand how SqueezeNet was able to reduce the number of parameters consider a convolutional layer formed only with 3×3 filters. Its numbers of parameters is:

$$(\#\text{ filters}) \cdot (\#\text{ input channels}) \cdot (3 \cdot 3)$$

So, to maintain a small number of parameters in a CNN it is important to reduce both the number of filters and the number of input channels. The authors of [7], following this reasoning, have employed three strategies:

1. Replace most of 3×3 filters with 1×1 filter: in this way they reduce the numbers of 3×3 filters by replacing them with 1×1 filters that have 9 times fewer parameters;

2. Decrease the number of input channels of 3 ×3 filters: to do so squeeze layers was used (similar but different from the squeeze phase described in secction 3.2.2);

3. Downsaple late in the network: in this way convolutional layers have large activation map leading to higher accuracy [12];

To implement strategy 3, max-pooling with a stride of 2 is performed after four layers while for the first two strategies, Fire modules are introduced.

3.3.1 Fire modules

A Fire module consists of:

- A squeeze convolution layer: it uses $s_{1\times1}$ filters 1×1 to squeeze all the input channels into $s_{1\times1}$ single channel feature maps (strategy 1); shown in Figure 12;
- An expand layer: formed by $e_{1\times1}$ filters 1×1 and $e_{3\times3}$ filters 3×3; it takes as input the $s_{1\times1}$ feature maps and returns the $e_{1\times1} + e_{3\times3}$ feature maps obtained from the convolutional filters of this layer. To limit the number of input channels of the 3×3 filters, Fire modules set $s_{1\times1}$ to be less than $e_{1\times1} + e_{3\times3}$ (strategy 2).

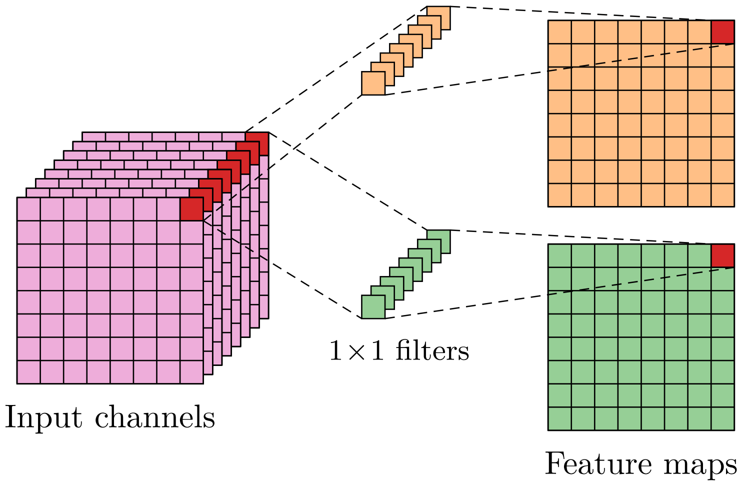

Figure 12: Squeeze convolutional layer with $s_{1\times1} = 2$ and 8 input channels.

By stacking multiple fire modules, SqueezeNet achieves a significant reduction in the number of parameters while maintaining competitive accuracy.

4 Hyperparameter optimization

Now that the models have been defined in the previous section, an Hyperparameter (HP) optimization phase is needed. The HP choice is essential to obtain the best performance of a neural network, and it can significantly influence the model's ability to generalize to unseen data.

Only 3 Hyperparameters [with related possible values] will be considered in the models:

1. Learning rate: the step size at each iteration while moving toward a minimum of the loss function; it controls how much the model weights are updated during training. $[0.0001, 0.1]$
2. Batch Size: the number of samples used to compute each update of the model weights; it influences the stability and speed of the training process. $\{2, 4, 8, 16, 32\}$
3. Dropout percentage: the fraction of input units to drop during training to prevent overfitting; it helps improve the model's ability to generalize. $[0, 0.7]$

Given the use of state-of-the-art architectures, other HPs such as activation functions, layers number, and layer types have not been changed.

4.1 Search Algorithms

The search for optimal HP values can be done in two ways: manually setting them based on prior experience or intuition, or using automated search algorithms.

Although manual testing remains one of the most widely used methods, especially in the student environment, it requires a deep knowledge of the used algorithms and their HPs meaning.

In addition, it has been found that a manual procedure is ineffective for several problems due to different factors, such as the high dimensionality of the hyperparameter space, the presence of complex interactions between hyperparameters, and the computational cost of exhaustively evaluating all possible combinations [13]. For this reasons, manual testing was discarded in favor of using a search algorithm.

Formally, a HPs optimization problem aims to find the HPs configuration $\boldsymbol{x}^* \in X$ such that:

$$\boldsymbol{x}^* = \underset{\boldsymbol{x} \in X}{\operatorname{argmin}} f(\boldsymbol{x})$$

where X is the search space containing all the possible hyperparameter configurations, and f is the objective function (e.g., validation loss or error) to be minimized. A search algorithm is an heuristic algorithm whose goal is to achieve the optimal, or near-optimal, HPs configuration within a given budget, which can be of time or computational resources.

In the next section the simplest search algorithms will be shown. Given the low number of HPs, the use of more sophisticated algorithms like Bayesan optimization or Gradient based optimization was not deemed necessary in favor of a simpler grid search.

4.1.1 Grid search

Grid search is the most straightforward hyperparameter tuning algorithm. It discretizes the search space of each hyperparameter and evaluates all possible combinations. The combination that yields the best average performance is selected as the optimal set of hyperparameters.

The well-performance regions cannot be detected, however, to identify the global optimums, the following procedure can be performed:

1. Start with a large search space and step size;
2. Narrow the search space and step size based on the best performance regions found on the previous step;
3. Repeat step 2 multiple times until an optimum is reached.

Grid search can be easily be implemented and parallelized but, being de facto brute force, it's ineffective with a high number of HPs. In fact, its computational complexity is $O(n^k)$ where k is the HPs number and n is the number of possible values.

4.1.2 Random search

Random search is a search algorithm similar to grid search. Instead of exploring all the values in the search space, random search randomly samples hyperparameter combinations from the defined ranges or sets. Each sampled configuration is evaluated, and the best-performing set is selected.

With a limited budget, random search is able to explore a larger search space than grid search by reducing the probability of wasting time on small poor-performance regions. Its computational complexity is $O(n)$, where n is the number of sampled configurations defined before the optimization process starts.

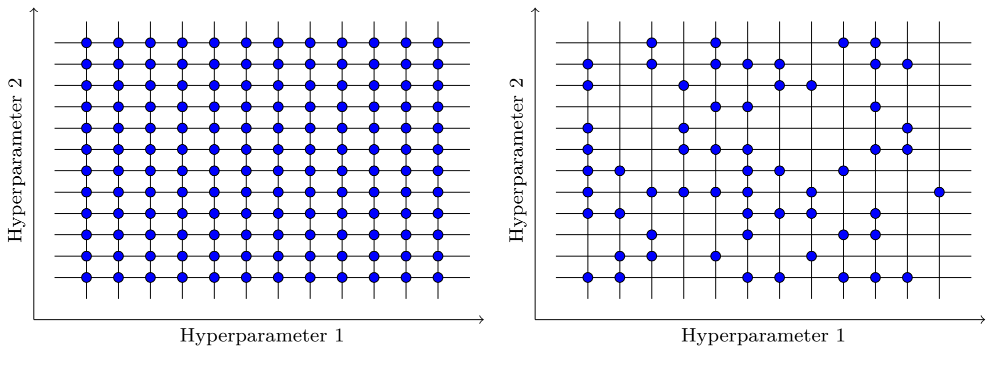

Figure 13: Comparison between grid search(left) and random search(right) with 2 HPs

The random search and grid space problem is that every evaluation is independent of previous evaluations; thus, they waste massive time evaluating poorly-performance areas of the search space.

4.2 Ray Tune

To perform the HPs optimization phase Ray Tune [14], an industrial standard tool for distributed HP tuning, was used. Ray Tune enables efficient parallelization and management of multiple training runs, making it suitable for a grid search strategy.

The used search spaces for every HP are:

1. Learning rate: the search was performed using `tune.loguniform` between $[0.0001, 0.1]$, which samples values x such that $\log x$ is uniformly distributed between $\log a$ and $\log b$;
2. Batch size: grid space between $\{2, 4, 8, 16, 32\}$ was used;
3. Dropout percentage: grid space between $[0, 0.7]$ was used dividing it into 35 equally spaced points.

To run every HPs configuration trial Asynchronous Successive Halving Algorithm (ASHA) [15] was used.

4.2.1 Asynchronous Successive Halving Algorithm

ASHA is an early stopping algorithm that efficiently allocates resources by quickly terminating underperforming trials and allocating more resources to promising ones.

More precisely, ASHA's procedure can be summarized as follows:

1. Launch many trials, one per HPs configuration;
2. Every trial is performed for at least g epochs;
3. After g epochs all trials that have reached the same amount of epochs are evaluated;
4. Only a $1/r$ fraction of the best trials are kept, while the others are early stopped;
5. The remaining trials continue to be performed and will be evaluated in the next round;
6. Only the trials that survive all rounds are trained until the maximum number of epochs or until convergence.

where g is the minimum number of epochs that each trial must run before being considered for early stopping, and r is the reduction factor determining the proportion of trials retained at each round.

This approach significantly reduces the computational cost compared to evaluating all configurations for the full number of epochs, while still identifying high-performing hyperparameter settings.

4.3 Results

For each models, 175 hyperparameter configurations were analyzed with a maximum of 10 epochs per configuration and a minimum of 3 epochs. Given the large number of trials, to evaluate every trial, it was preferred not to use Cross Validation in favor of a validation set.

The results of HPs optimization are shown in Table 1.

	AlexNet	**MobileNetV3**	**SqueezeNet**
Learning rate	2.6062×10^{-4}	2.5266×10^{-4}	2.8481×10^{-2}
Batch size	8	32	32
Dropout perc.	0.0412	0.3089	0

Table 1: HPs optimization results

It is noteworthy that the HPs optimization process found that SqueezeNet achieves its best performance without dropout. This suggests that, for this architecture, regularization through dropout is detrimental given the small size of the model.

To obtain this results, each model require, more or less, 2 hours with an NVIDIA RTX 3090 as GPU and an Intel Core i7-12700K as CPU.

5 Training

In this section, the training phase of the model is described. Mean Squared Error (MSE) was chosen as the loss function; other functions like MAE or Huber loss were tested but showed lower performance than MSE. k-fold Cross Validation (CV) was used to estimate the generalization performance and to mitigate overfitting. For each fold, the model was trained on $k-1$ subsets and validated on the remaining one, rotating through all folds. This approach ensures that each data point is used for both training and validation, providing a more robust evaluation of model performance. A k value of 5 was used.

Each model was trained from scratch through the ADAM optimizer [16] with the hyperparameters found in the previous section.

5.1 ADAM

Adam combines the strengths of two other popular optimizers:

- Stochastic Gradient Descent (SGD) with momentum: updates the parameters by combining the current gradient with an exponential average of past gradients, so as to accelerate optimization in consistent directions and reduce oscillations.
- RMSProp: it dynamically adjusts the learning rate of each parameter by dividing the gradient by an average of its recent magnitudes, improving stability and convergence especially with noisy or differently scaled gradients.

This combination of techniques allows ADAM to be more efficient and robust on noisy datasets.

5.2 Training results

The results are shown in Table 2. A Pareto dominance of MobileNetV3 is evident, as it achieves, by far, the lowest loss values across all metrics compared to the other models. This indicates that MobileNetV3 outperforms, in terms of accuracy, both AlexNet and SqueezeNet. This superiority was to be expected since AlexNet is an obsolete model compared to MobileNetV3 and SqueezeNet focuses more on model reduction than accuracy.

	Train loss	**Valid. Loss**	**Test Loss**	**5−CV Loss**
AlexNet	0.03394	0.03214	0.03476	0.03489
MobileNetV3	0.00707	0.01499	0.01450	0.01362
SqueezeNet	0.03703	0.03447	0.03849	0.03682

Table 2: Models performances

Regarding efficiency, in terms of inference time and model size (Table 3), as expected, SqueezeNet is the most lightweight model, achieving the smallest size and fast inference, especially on GPU, maintaining similar accuracy to the much heavier AlexNet.

While not as small as SqueezeNet, MobileNetV3 is significantly more compact and faster than AlexNet, and it achieves the best accuracy among all models.

Inference times were calculated by averaging the individual datapoint inference times over the testset.

	tCPU(ms)	**tGPU(ms)**	**Size(MB)**
AlexNet	5.3542	0.2395	142.29
MobileNetV3	3.3972	0.8319	5.85
SqueezeNet	3.8984	0.3311	2.76

Table 3: Models inference times with CPU and GPU and size

6 Conclusion

References

[1] Cyrill Stachniss, John J. Leonard, and Sebastian Thrun. "Simultaneous Localization and Mapping". In: *Springer Handbook of Robotics*. Ed. by Bruno Siciliano and Oussama Khatib. Cham: Springer International Publishing, 2016, pp. 1153–1176. ISBN: 978-3-319-32552-1. DOI: 10.1007/978-3-319-32552-1_46. URL: https://doi.org/10.1007/978-3-319-32552-1_46.

[2] Rainer Kümmerle et al. "On Measuring the Accuracy of SLAM Algorithms". In: *Autonomous Robots* 27 (Nov. 2009), pp. 387–407. DOI: 10.1007/s10514-009-9155-6.

[3] Matteo Luperto, Valerio Castelli, and Francesco Amigoni. "Predicting Performance of SLAM Algorithms". In: *CoRR* abs/2109.02329 (2021). arXiv: 2109.02329. URL: https://arxiv.org/abs/2109.02329.

[4] Matteo Luperto and Francesco Amigoni. "Extracting Structure of Buildings Using Layout Reconstruction". In: *Intelligent Autonomous Systems 15*. Ed. by Marcus Strand et al. Cham: Springer International Publishing, 2019, pp. 652–667. ISBN: 978-3-030-01370-7.

[5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012.

[6] Andrew Howard et al. *Searching for MobileNetV3*. 2019. arXiv: 1905.02244 [cs.CV]. URL: https://arxiv.org/abs/1905.02244.

[7] Forrest N. Iandola et al. *SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size*. 2016. arXiv: 1602.07360 [cs.CV]. URL: https://arxiv.org/abs/1602.07360.

[8] URL: https://alexlenail.me/NN-SVG/AlexNet.html.

[9] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[10] URL: https://eli.thegreenplace.net/2018/depthwise-separable-convolutions-for-machine-learning/.

[11] Jie Hu et al. *Squeeze-and-Excitation Networks*. 2019. arXiv: 1709.01507 [cs.CV]. URL: https://arxiv.org/abs/1709.01507.

[12] Kaiming He and Jian Sun. *Convolutional Neural Networks at Constrained Time Cost*. 2014. arXiv: 1412.1710 [cs.CV]. URL: https://arxiv.org/abs/1412.1710.

[13] Li Yang and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice". In: *Neurocomputing* 415 (2020), pp. 295–316. ISSN: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2020.07.061. URL: https://www.sciencedirect.com/science/article/pii/S0925231220311693.

[14] Richard Liaw et al. "Tune: A Research Platform for Distributed Model Selection and Training". In: *arXiv preprint arXiv:1807.05118* (2018).

[15] Liam Li et al. *A System for Massively Parallel Hyperparameter Tuning*. 2020. arXiv: 1810.05934 [cs.LG]. URL: https://arxiv.org/abs/1810.05934.

[16] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.